

IMP Project Documentation

Š01 - ESP32: Světelná tabule

Rastislav Uhliar (xuhliar00)

November 2025

Table of Contents

1	Introduction	2
1.1	Demonstration video link	2
1.2	Components	2
2	Implementation	3
2.1	Environment	3
2.2	Framebuffer structure	3
2.3	Text Rendering	3
2.4	Time Multiplexing	3
2.5	LED Matrix	4
2.5.1	Column Selection	4
2.5.2	PWM Control	4
2.6	Scrolling	5
2.7	Buttons	6
3	Conclusion	7

1 Introduction

The goal of this project was to create program for displaying scrolling text on a Analog RGB LED Shield module on a development board based on SoC ESP32 from Espressif.

1.1 Demonstration video link

<https://nextcloud.fit.vutbr.cz/s/8zNNabdkj4Bk5Bt>

1.2 Components

- **Wemos D1 R32** - development board
- **Analog RGB LED Shield** - RGB LED matrix
- **74HCT154** - 4-to-16 decoder
- **TLC5947** - 24-channel PWM controller
- **4× buttons** - device control

2 Implementation

2.1 Environment

The project was implemented in Arduino IDE.

2.2 Framebuffer structure

```
// Stores RGB values for each pixel in the matrix  
// 0=Red, 1=Blue, 2=Green  
uint16_t frame[COLS][ROWS][3];
```

The LEDs are in order Blue, Red, Green. Each color is stored as a 12-bit value (0-4095) which specifies the brightness. In the framebuffer (frame) i stored the colors in ther RGB order therefore it is remapped when sending the data to TLC.

```
// For each row in the current column  
// Populate data array with color values mapped to TLC channels  
for (int r = 0; r < ROWS; r++) {  
    // The wiring is Blue, Red, Green  
    data[r * 3 + 0] = frame[scanCol][r][1]; // Channel offset 0 Blue  
    data[r * 3 + 1] = frame[scanCol][r][0]; // Channel offset 1 Red  
    data[r * 3 + 2] = frame[scanCol][r][2]; // Channel offset 2 Green  
}
```

2.3 Text Rendering

A bitmap font is used for each character A-Z and space. Each byte represents one column and each bit represents a pixel that is either turned on or turned off.

```
// Font with letters 5x7 pixels  
const uint8_t font[][5] = {  
    {0x7E, 0x11, 0x11, 0x11, 0x7E}, // A  
    ...  
}
```

Function drawCharacter is used to draw a single character at position x. This function is used in drawText that draws the whole message to the frame.

2.4 Time Multiplexing

The display can show just one column at time. Since the columns are switched quickly it creates an illusion and we see the whole message normally.

For each column the program waits 1.25ms before drawing the next one. Since there are 16 columns that is $16 * 1.25 = 20\text{ms}$. The whole display is refreshed every 20ms that is $1/0.02\text{s} = 50\text{Hz}$. This is handled in refreshDisplay function that is called in the loop part of the program.

2.5 LED Matrix

2.5.1 Column Selection

The 74HCT154 decoder converts a 4-bit address and activates one of the 16 outputs. The output then switches a P-channel MOSFET transistor that supplies power to the selected column of the matrix.

```
// Select active column using 4 to 16 decoder after that P channel MOSFET supplies power to the
void selectColumn(int c) {
    digitalWrite(ADDR_A, (c >> 0) & 1); // A0 (LSB)
    digitalWrite(ADDR_B, (c >> 1) & 1); // A1
    digitalWrite(ADDR_C, (c >> 2) & 1); // A2
    digitalWrite(ADDR_D, (c >> 3) & 1); // A3 (MSB)
}
```

2.5.2 PWM Control

The TLC5947 controls the brightness of the individual LEDs using PWM with 12-bit values (0-4095). In each column there are 8 rows with 3 values for Red, Green, Blue. That is $8 * 3 = 24$ channels. For each channel there are 12-bits that is 288bits which is equal to the to the shift register that TLC5947 is equipped with.

```
// Sends PWM values to TLC5947
void sendTLC(uint16_t *v) {
    // 24 channels x 12 bits each = 288 bits
    // 288 bits / 8 = 36 bytes
    uint8_t buf[36];
    memset(buf, 0, sizeof(buf));

    int totalBitPosition = 0;
    // Iterate over channels in reverse order
    for (int channel = CHANNELS - 1; channel >= 0; channel--) {
        // Uint16_t is 16bits therefore mask the lower 12 bits the 4 bits are not needed
        uint16_t pwmValue = v[channel] & 0xFFFF;

        // Go through each bit of the 12-bit pwmValue from MSB to LSB
        for (int bitNumber = 11; bitNumber >= 0; bitNumber--) {
            // Calculate byte and bit position
            int byteIndex = totalBitPosition / 8;
            // 7 - bit position because MSB is sent first
            int bitOffset = 7 - (totalBitPosition % 8);

            // Extract the bit value by shifting and masking
            uint8_t bitValue = (pwmValue >> bitNumber) & 1;
            // Set the bit in the buffer
            buf[byteIndex] |= (bitValue << bitOffset);

            // Advance to next bit position
            totalBitPosition++;
        }
    }
}
```

```

    }
}

// Sets latch pin low before data transfer
digitalWrite(TLC_LATCH, LOW);
// SPI configuration 8Mhz clock speed
SPI.beginTransaction(SPISettings(8000000, MSBFIRST, SPI_MODE0)); // 8 MHz < 30 MHz max
// Send data via SPI
SPI.writeBytes(buf, sizeof(buf));
// End SPI transaction
SPI.endTransaction();
// Latch the data into the PWM outputs
digitalWrite(TLC_LATCH, HIGH);
// Wait to ensure that the data is latched
delayMicroseconds(1);
// Set latch low again
digitalWrite(TLC_LATCH, LOW);
}

```

2.6 Scrolling

Text moves pixel by pixel from right to left. This is handled in updateScrollingText function.

```

// Refresh display one column at a time
// After 1.25ms switches to the next column
// that means 16*1.25ms = 20ms the display has whole new "image" every 50Hz
void refreshDisplay() {
    uint32_t now = micros();
    // Check if it's time to switch to the next column
    if ((uint32_t)(now - lastColumnTime) >= WAIT) {
        uint16_t data[CHANNELS];

        // For each row in the current column
        // Populate data array with color values mapped to TLC channels
        for (int r = 0; r < ROWS; r++) {
            // The wiring is Blue, Red, Green
            data[r * 3 + 0] = frame[scanCol][r][1]; // Channel offset 0 Blue
            data[r * 3 + 1] = frame[scanCol][r][0]; // Channel offset 1 Red
            data[r * 3 + 2] = frame[scanCol][r][2]; // Channel offset 2 Green
        }

        // Blank display during the switch
        digitalWrite(TLC_BLANK, HIGH);
        // Select the current column
        selectColumn(scanCol);
        // Send data for this column
        sendTLC(data);
        // Enable outputs
        digitalWrite(TLC_BLANK, LOW);
    }
}

```

```

    // Save time of this column update and move to the next column
    lastColumnTime += WAIT;
    scanCol++;
    // Wrap around after column 15
    if (scanCol >= COLS) scanCol = 0;
}
}

```

2.7 Buttons

The buttons are used to control the device.

Button debounce is handled by checking if there was 200ms between last press. Without this the speed for example jumps through the levels and doesn't go over them one after another.

```

// To avoid button mechanical bounce there should be 200ms between presses
bool isButtonPressed(int btnPin) {
    uint32_t now = millis();
    // If button is pressed and debounce time has passed
    if (digitalRead(btnPin) == LOW && (now - lastBtnMs >= 200)) {
        // Update last button press time and return true
        lastBtnMs = now;
        return true;
    }
    return false;
}

```

Button 1 Switches between 5 text messages - “PROJECT DEMO”, - “IMP”, - “FIT”, - “HELLO”, - “MEOW”

Button 2 Changes the scrolling speed from slow to fast (5 levels)

Button 3 Changes the text color from red to green to blue to white

Button 4 Changes the text brightness from dim to bright (4 levels)

3 Conclusion

The project implemented all required functions. Scrolling text is displayed using time multiplexing and the user can control the display using 4 buttons.